

CODEXLABZ TECHNOLOGIES

PROJECT WORK DIARY

Salesforce CRM Track | Property Tax Management System

Project	Property Tax Management System (PTMS)
Platform	Salesforce (CRM + Experience Cloud + LWC + Agentforce)
Duration	21 Weeks Academic Year 2025–2026

TEAM MEMBERS

OMKAR

Salesforce Backend Developer

Apex · LWC · Admin · Integration · Agentforce

KAUSTUBH

Frontend + Integration Developer

LWC · Experience Cloud UI · CSS · Leaflet.js

Weeks 1–5 Learning Phase

Weeks 6–19 Project Phase

Weeks 20–21 Final Evaluation

Project Overview

This Project Work Diary documents the week-by-week progress of the Industry Internship & Project undertaken at **Codexlabz Technologies**. The project — the **Property Tax Management System (PTMS)** — is built on the Salesforce platform and simulates the digital infrastructure of a municipal corporation's property tax department, modelled on the Greater Chennai Municipal Corporation (GCMC).

System Modules

Module	Technology	Owner
Custom Objects (Property, Owner, Assessment, Payment)	Salesforce Schema	Omkar
eKYC Verification Flow	Screen Flow + Apex	Omkar + Kaustubh
Property Registration + Approval Workflow	Screen Flow + Approval Process	Omkar + Kaustubh
BSR-Based Tax Calculation	Apex Trigger (PropertyTrigger)	Omkar
Tax Payment Record Auto-Creation	Apex Trigger (TaxAssessmentTrigger)	Omkar
Razorpay Payment Gateway Integration	Visualforce Bridge + Apex + LWC	Omkar + Kaustubh
Citizen Dashboard (Property Cards + Summary)	LWC + Apex Controller	Omkar + Kaustubh
Cases & Grievances Module	LWC + Apex + Salesforce Cases	Omkar + Kaustubh
Zone & Ward Finder (Map)	Leaflet.js + OpenStreetMap + Vercel	Kaustubh
Agentforce AI Assistant	Agentforce + Knowledge Articles	Omkar + Kaustubh
Scheduled Reassessment & Email Reminders	Schedulable + Batchable Apex	Omkar
Experience Cloud Portal Setup	LWR Template + Experience Builder	Kaustubh

21-Week Schedule Summary

Week	Module / Milestone	Phase
1	Orientation + Foundations — Agile, HTML/CSS/JS, Data Modelling, Salesforce org setup	Learning
2	Salesforce Core Part I — Lightning UI, Profiles, Standard Objects, Trailhead	Learning
3	Salesforce Core Part II — Security model, OWDs, Custom Objects, Relationships	Learning
4	Salesforce Core Part III — Formulas, Validation Rules, Approval Processes, Flows	Learning
5	Sales / Service / Experience Cloud — Cases, Knowledge, Portal site creation	Learning
6	Hackathon Phase 1 & MFRP-1 — Data model built, Registration Flow, Sharing Sets	Project
7	Apex Basics Part I — SOQL, DML, without sharing, Dashboard controller skeleton	Project
8	Apex Basics Part II — Wrapper classes, bulkification, MakePaymentController	Project
9	Apex Triggers & Tests — PropertyTrigger (BSR tax), test coverage, propertyCard LWC	Project
10	Visualforce & MVC — RazorpayPayment VF bridge page, LWC connectedCallback params	Project
11	Async Apex & Deployment — Scheduler, Batch, Change Sets, loading spinner UX	Project
12	Integration — Two-step payment confirmation, full Razorpay end-to-end test	Project
13	Modern JavaScript & LWC Foundations — ES6+, Dashboard LWC, responsive grid	Project
14	LWC Advanced — Payment, eKYC components, NavigationMixin, CustomEvent	Project

15	Cases, Grievances & TaxAssessment Trigger — CaseController, caseGrievance LWC	Project
16	Experience Cloud Deep-Dive — Portal pages, audiences, Zone & Ward Finder	Project
17	Agentforce & AI Assistant — Knowledge Articles, Agent Topics, chat widget	Project
18	DevOps & Git — Salesforce DX, GitHub, branch strategy, pull request review	Project
19	Testing & Bug Fixes — Test coverage 82%, data isolation bugs fixed, UI bugs fixed	Project
20	Consolidation & MFRP Final — Full regression test, demo, report draft	Project
21	Final Evaluation & Project Demonstration — Complete demo, report submission	Project

Week 01 | Orientation + Foundations · LEARNING PHASE — No Project Work

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- First day at Codexlabz Technologies. Attended orientation with mentor team. Understood internship schedule, team roles and expectations.
- Installed VS Code, Salesforce CLI (sf), Node.js and set up first Salesforce Developer Edition org.
- Covered Software Development Life Cycle — Waterfall vs Agile vs Scrum. Understood sprint planning, daily stand-ups and retrospectives.
- Learned HTML and CSS fundamentals, JavaScript basics (variables, functions, DOM), and JSON structure.
- Covered conceptual and logical data modelling — ER diagrams, normalisation, primary and foreign key relationships.
- Completed foundational assessment on Agile concepts and data modelling at end of week.

■ Project Work & Progress

No project work this week. Completed environment setup. Successfully logged into Salesforce Developer Edition org and explored the standard objects and Setup menu for the first time.

■ Challenges Faced

Setting up Salesforce CLI was new. The sf login web command opened the browser but terminal was not detecting successful login. Fixed by running the correct callback and re-authenticating with correct flags.

■ Additional Comments

Agile methodology sessions were insightful. Understanding sprints and story points early will help plan project phases. Excited to start Salesforce-specific modules next week.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Attended orientation. Understood roles: Omkar handles Salesforce backend (Apex, objects, automation), I handle frontend (LWC, Experience Cloud UI, CSS).
- Installed VS Code with Salesforce Extension Pack. Set up Node.js and Git.
- Studied HTML semantics and CSS layout — flexbox and grid. Built a simple static page to practise.
- Learned JavaScript basics: functions, event listeners, fetch API for JSON data.
- Explored how Salesforce Lightning components differ from standard HTML pages — components are self-contained modules.

■ Project Work & Progress

No project this week. Built a static HTML mock-up of what the citizen dashboard might look like — property cards in a grid with Bootstrap-style layout. Used this as a reference wireframe going forward.

■ Challenges Faced

JavaScript event listeners were confusing at first — attaching an event to a button that did not yet exist in the DOM. Learned about DOMContentLoaded to wait for the page to be ready before attaching events.

■ Additional Comments

Creating the wireframe early was a good decision. Having a visual target makes the LWC work feel concrete. Will refine after seeing how Experience Cloud pages are structured.

Week 02 | Salesforce Core — Part I · LEARNING PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- On-boarded to the Salesforce track. Explored the Salesforce Lightning UI — App Launcher, Setup menu, Object Manager.
- Configured company settings, fiscal year and business hours in the Developer Edition org.
- Learned about Users, Profiles and Permission Sets — profile as minimum access, permission set as additional keys.
- Studied Standard Objects: Account, Contact, Lead, Opportunity. Understood relationships between them.
- Completed Trailhead badges: 'Salesforce Platform Basics' and 'Data Modeling'.

■ Project Work & Progress

No project work yet. Explored the Object Manager in the org. Noted that Property__c, Property_Owner__c, Tax_Assessment__c and Tax_Payment__c will be created as custom objects in a future week.

■ Challenges Faced

The Setup menu in Salesforce has hundreds of options. Building a mental map of where things live — Users under Admin, Objects under Object Manager — took time but helped navigation significantly.

■ Additional Comments

The difference between a Profile and a Permission Set became clear: profile is the door, permission set is extra keys. This distinction is critical when configuring the citizen portal user later.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Studied Salesforce Lightning Experience layout — navigation bar, App Builder, record pages.
- Explored how standard objects like Contact relate to custom ones — Property Owner will link to a Contact.
- Studied Profiles in detail — Customer Community User profile will restrict what citizens see on the portal.
- Completed Trailhead badge: 'User Management'.

■ Project Work & Progress

Sketched updated wireframes for the Experience Cloud portal — login page, registration page, dashboard and payment page. Shared with Omkar for alignment on what data fields will be needed per page.

■ Challenges Faced

Understanding the difference between internal Salesforce users and Experience Cloud portal users took a second read. Internal users have full Lightning access; portal users only see the Experience Cloud site.

■ Additional Comments

Wireframes are now Salesforce-specific — instead of generic cards, they show field labels like Property Name, Zone, Ward, Annual Tax. Alignment with the data model is important before building.

Week 03 | Salesforce Core — Part II · LEARNING PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Deep-dived into Salesforce security model: Organisation-Wide Defaults (OWD), Role Hierarchy, Sharing Rules and Manual Sharing.
- Understood the layered security model: OWD is most restrictive default, sharing rules open access, manual sharing handles one-off exceptions.
- Learned custom objects and fields: text, number, currency, date, picklist, lookup and master-detail relationship types.
- Created practice custom object (Student__c) with various field types to understand Schema Builder.
- Learned about self-referential lookups and many-to-many junction objects.

■ Project Work & Progress

Designed the four custom objects for the project: Property__c, Property_Owner__c, Tax_Assessment__c, Tax_Payment__c. Drew the ER diagram. Shared with mentor — approved with minor field additions.

■ Challenges Faced

Difference between Lookup and Master-Detail: Master-Detail creates tight parent-child coupling (child cannot exist without parent), while Lookup is loose. Decided to use Master-Detail for Tax_Assessment__c to Tax_Payment__c.

■ Additional Comments

Getting OWDs right is critical for the portal. If OWDs are too open, all portal users will see each other's property data — a serious data privacy issue. Setting OWD to Private for all four objects is the correct starting point.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Studied Salesforce sharing model from a portal perspective. Sharing Sets are the key mechanism for Experience Cloud users to see their own records.
- Learned about Record Types — useful if different page layouts are needed for different property types in future.
- Studied field-level security — PAN and Aadhar fields should have restricted FLS so only authorised roles can view them.
- Completed Trailhead badge: 'Data Security'.

■ Project Work & Progress

Updated wireframes to include the eKYC form — two steps: personal info then PAN and Aadhar. Designed the property registration form layout. Sent to Omkar for field mapping confirmation.

■ Challenges Faced

Understanding Sharing Sets in Experience Cloud — they work differently from internal sharing rules. The mapping User.Contact = Record.Related_Contact is how portal users get access to their own records.

■ Additional Comments

FLS for PAN and Aadhar fields needs careful configuration. Citizens write these fields; officers reviewing registrations should see them; general internal users should not.

Week 04 | Salesforce Core — Part III · LEARNING PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Learned Formula Fields — calculated fields that reference other field values. Annual Tax on Tax_Assessment__c will use a formula.
- Studied Validation Rules — declarative rules preventing bad data. Applied to ensure built-up area is positive and PAN format is correct.

- Learned Approval Processes in depth — entry criteria, approval steps, approver assignment and approval/rejection actions.
- Built a practice approval process for leave requests to understand the mechanism before applying to property registration.
- Learned Flows — Screen Flow for multi-step user input, Record-Triggered Flow for automation after record changes.

■ Project Work & Progress

Designed the two-level approval process for property registration on paper: Step 1 — Clerk Review, Step 2 — Tax Officer Approval. Identified field state transitions: Pending Clerk Review → Pending Officer Review → Active.

■ Challenges Faced

Approval processes lock records once submitted. This is a critical constraint for the payment flow. Mentor flagged it early — the payment confirmation will need to save transaction data BEFORE triggering the approval. This became the two-step payment confirmation design.

■ Additional Comments

Record-locking behaviour of approval processes was an unexpected discovery. Documenting it early helped plan the payment module architecture in advance rather than encountering it as a bug later.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Studied Salesforce Reports and Dashboards — will be used to build the officer-facing analytics view.
- Learned Screen Flows — how they present multi-step forms to users. This is how the property registration form will work on the portal.
- Explored Flow Builder UI — Decision elements, Assignment elements, Create Record, Update Record and Screen elements.
- Completed Trailhead badge: 'Approval Processes'.

■ Project Work & Progress

Designed the property registration Screen Flow step by step — Step 1: property type and basic details, Step 2: address and zone/ward, Step 3: built-up area details. Drew the full flow diagram and shared with Omkar.

■ Challenges Faced

Mapping Screen Flow steps to wireframe pages was straightforward, but understanding how the Flow creates a record and then a second flow triggers the approval process required careful thought about event chaining.

■ Additional Comments

The Screen Flow approach means the citizen registration form is built declaratively — no Apex needed for the registration screen itself. This is the declarative-first principle in practice.

Week 05 | Sales, Service & Experience Cloud · LEARNING PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Studied Service Cloud: Cases object, case assignment rules, escalation rules, web-to-case and email-to-case setup.
- The grievances module will use Salesforce's native Cases object — citizens raise cases through the portal, officers handle them in Lightning.
- Got full introduction to Experience Cloud — creating a digital experience site, choosing templates (LWR vs Aura) and configuring pages.

- Learned about Experience Cloud user licences: Customer Community, Customer Community Plus, External Identity. Project uses Custom External Identity User profile.
- Studied Lightning Knowledge — Articles that the Agentforce AI assistant will use for property tax queries.

■ Project Work & Progress

Created the Experience Cloud site in the Developer Edition org using the Build Your Own (LWR) template. Site URL configured. Verified site loads correctly. This is the shell that all portal pages will live in.

■ Challenges Faced

Experience Cloud site was not activating — error about Guest User Profile settings. Had to enable correct settings in Digital Experiences setup and configure Guest User OWD correctly. Took about an hour.

■ Additional Comments

Service Cloud Cases are a natural fit for the grievances module. Using native Cases means the officer sees citizen grievances in the same interface as all other work — no separate system needed.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Studied Experience Cloud page builder — drag-and-drop components for LWR sites. Difference between standard components and custom LWC components.
- Learned how to add custom LWC components to Experience Cloud pages using App Builder.
- Studied Experience Builder navigation menus and how to create page links in the portal.
- Completed Trailhead badge: 'Experience Cloud Basics'.

■ Project Work & Progress

Designed the full portal navigation: Home (Dashboard) → Register Property → eKYC → Make Payment → Zone & Ward Finder → Raise Grievance. Added placeholder pages in Experience Builder for each section.

■ Challenges Faced

LWR template has fewer standard components than the Aura template. Some components seen in tutorials were not available for LWR. Learned to rely on custom LWC components for most UI needs.

■ Additional Comments

The navigation structure is clear now. The Home page will be the propertyOwnerDashboard LWC — the most complex component. Planning it well early avoids redesign later.

Week 06 | Hackathon Phase 1 & MFRP-1 · PROJECT PHASE BEGINS

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Skills Applied

- Four-day hackathon using Agile Scrum sprints. Daily stand-ups with mentor review at end of each day.
- Created the four custom objects in the org: Property__c, Property_Owner__c, Tax_Assessment__c, Tax_Payment__c with all planned fields.
- Configured OWDs: Property — Private, Tax_Assessment — Private, Tax_Payment — Private.
- Created Sharing Sets for Tax_Assessment__c and Tax_Payment__c — mapping User.Contact = Property.Owner_Contact__c.
- Built the Screen Flow for property registration with three steps. Tested in the portal — form collects data and creates the Property__c record.

■ Project Work & Progress

End-of-hackathon demo: showed property registration flow working in the Experience Cloud portal. Property record created in Salesforce with all fields populated correctly. Approval process submitted and appeared in approver's queue. MFRP-1 received positive review from SME panel.

■ Challenges Faced

Sharing Set was not giving portal users access to Tax Assessment records. Root cause: the mapping was referencing the wrong field path. Correct path is Tax_Assessment__c → Property__c → Owner_Contact__c. Took two hours to identify.

■ Additional Comments

The hackathon forced integration of all learning from weeks one through five into one working system. End-to-end thinking was a step up from isolated exercises. MFRP-1 review gave good confidence to continue.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Skills Applied

- Built the eKYC form component as a static HTML mock-up first, then began converting to an LWC.
- Designed the property card layout — each card shows property name, door number, type, zone, ward, area and a payment status badge.
- Worked with Omkar to agree on the @AuraEnabled methods needed — drafted a list of what each component will call.

■ Project Work & Progress

Produced static HTML and CSS of the eKYC form and property card layout. Property card design was presented during MFRP-1 demo alongside Omkar's backend work. Panel appreciated the clean visual design.

■ Challenges Faced

CSS in LWC is scoped — styles in one component do not bleed into child components. This broke property card styling when nested inside the dashboard. Had to move shared styles to a static resource.

■ Additional Comments

The design review from the SME panel was very useful. One suggestion: add colour-coded badges for payment status (Pending = orange, Paid = green, Overdue = red). Incorporated into the property card design.

Week 07 | Apex Basics — Part I · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Started formal Apex training. Apex is Salesforce's Java-like language for programmatic logic on the platform.
- Covered data types, sObjects, SOQL (Salesforce Object Query Language), SOSL and DML operations: insert, update, delete, upsert.
- Learned Apex debugging with System.debug() and the Developer Console log viewer.
- Completed interim evaluation on Apex basics — SOQL queries, DML and basic class structure.
- Wrote first Apex class: a utility to fetch property details given a property ID.

■ Project Work & Progress

Wrote the skeleton of PropertyOwnerDashboardController.cls with without sharing keyword (required for portal users). Implemented getDashboardSummary() method returning total properties, pending count and tax due totals using SOQL aggregate functions.

■ Challenges Faced

SOQL aggregate functions (COUNT, SUM) return AggregateResult objects, not normal sObject types. Had to use result.get('expr0') to extract values. The syntax is less readable than expected but works correctly.

■ Additional Comments

The without sharing keyword is critical for all controllers serving portal users. Internal Salesforce sharing rules are too restrictive for External Identity users — without sharing bypasses them so Apex logic itself controls data access.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Attended Apex basics sessions to understand what Omkar is building — important for integrating LWC and Apex.
- Learned how @AuraEnabled annotation makes an Apex method callable from LWC. Understood cacheable=true versus non-cacheable.
- Studied the structure of an LWC component: HTML template, JavaScript controller, CSS file and meta.xml configuration.
- Practised building a simple hello-world LWC and deploying it to the org using VS Code Salesforce extension.

■ Project Work & Progress

Built the eKYCForm LWC HTML template with two steps: Step 1 has personal information fields (name, DOB, address), Step 2 has PAN and Aadhar number inputs. Navigation between steps uses a JavaScript flag (currentStep).

■ Challenges Faced

LWC template conditionals use if:true and if:false (older) or the newer lwc:if. The org version determines which applies. Tested both — used lwc:if as the modern standard supported in the org version.

■ Additional Comments

Two-step form design is better UX than a single long form. The citizen can focus on one section at a time. Will apply this progressive disclosure pattern to other forms too.

Week 08 | Apex Basics — Part II · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Advanced SOQL: relationship queries, aggregate functions with GROUP BY, SOQL in loops (governor limit violation) and best practices for bulkification.
- Learned Apex Collections: List, Set and Map — used to process multiple records efficiently in triggers.
- Studied SOSL — Salesforce Object Search Language for text-based search across multiple objects.
- Covered @AuraEnabled wrapper classes — inner classes with @AuraEnabled fields for clean LWC-Apex data contracts.
- Implemented PaymentDetailsWrapper inner class in MakePaymentController: baseAmount, discount, penalty, netPayable, dueDate, isOverdue.

■ Project Work & Progress

MakePaymentController.cls skeleton completed. getPaymentDetails() method returns wrapper with base amount, 2% discount if before due date, 10% penalty if overdue, and net payable. All methods use without sharing keyword.

■ Challenges Faced

Apex wrapper classes need @AuraEnabled annotation at class level to be serialisable to JSON for LWC. Forgot the annotation on inner class initially — Apex method returned null on LWC side. Adding @AuraEnabled to wrapper class

fields fixed it.

■ Additional Comments

Wrapper classes make the Apex-LWC contract explicit and stable. If Apex returns raw SOQL results, any schema change breaks the LWC. A wrapper class insulates the LWC from internal Apex changes.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Practised `async/await` in JavaScript — how calling an Apex method from LWC uses `await` since it returns a Promise.
- Learned destructuring to cleanly extract fields from Apex results: `const { baseAmount, discount, penalty } = result.`
- Studied LWC error handling: try-catch around imperative Apex calls, showing error messages using `lightning-formatted-text`.
- Learned `JSON.stringify()` for debug logging in LWC — helps inspect full objects in the browser console.

■ Project Work & Progress

Added error handling to all LWC components. A reusable `errorMessage` property and conditional HTML block show error banners when Apex calls fail. Tested by deliberately breaking the Apex method name — error banner appeared correctly.

■ Challenges Faced

LWC error objects from failed Apex calls have a `body.message` structure, not just a `message` property. Using `error.message` directly showed `undefined`. Correct pattern: `const msg = error.body?.message || 'Unknown error'.`

■ Additional Comments

Building error handling into every component from the start means users always see a helpful message instead of a silent failure. This is a habit worth forming early.

Week 09 | Apex Triggers & Test Classes · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Learned Apex Triggers — code that runs before or after DML events on records.
- Understood trigger context variables: `Trigger.new`, `Trigger.old`, `Trigger.newMap` for bulk-safe processing.
- Studied recursive trigger prevention using a static Boolean flag to prevent a trigger calling itself.
- Learned test classes: `@isTest`, `Test.startTest()/stopTest()`, `System.assert` and the 75% code coverage requirement.

■ Project Work & Progress

- Built `PropertyTrigger.cls`: fires after update when `Property_Status__c` changes to `Active`. Creates `Tax_Assessment__c` with BSR-based calculation. BSR rates: Individual House = 1.7, Apartment = 4.0, Commercial = 8.0, Government = 1.0.
- Annual Tax formula = $\text{BSR} \times \text{Built-Up Area} \times \text{Combined Tax Rate} \times \text{Annualizing Factor} / 100$. Due date set to ten days from assessment date.
- Wrote test class for `PropertyTrigger` — 88% code coverage achieved.

■ Challenges Faced

Writing bulk-safe triggers requires iterating `Trigger.new` with a List, not querying inside the loop. First version queried `Tax_Assessment__c` inside the loop — hit governor limits immediately in testing. Refactored to collect all property IDs first and query once.

■ Additional Comments

The PropertyTrigger is the heart of the tax calculation logic. Getting the BSR calculation right required re-reading the Greater Chennai Municipal Corporation formula carefully. Pro-rating for mid-year registrations was noted as a future enhancement.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Built propertyCard LWC — the individual property display card used inside the dashboard.
- Card shows: property name, door number, property type, zone, ward number, built-up area, annual tax, due date and payment status badge.
- Learned LWC @wire decorator to call Apex methods reactively and when to use imperative calls (for bypassing cache).
- Payment status badge uses CSS class applied via JavaScript getter: get badgeClass() returns the correct CSS class based on status.

■ Project Work & Progress

propertyCard LWC is functional. Renders all property details correctly. Status badge shows correctly: Pending (orange), Pending Verification (blue), Paid (green). The Pay Now button is conditionally shown using a getter — hidden when status is Paid or Pending Verification.

■ Challenges Faced

LWC getter for conditional CSS must return a class string, not a boolean. Initially tried using if:true with a boolean but CSS class toggling needs the string approach: `get badgeClass() { return this.status === 'Paid' ? 'badge-green' : 'badge-orange'; }`

■ Additional Comments

The property card is the most visible component in the whole portal. Getting it right visually is important. Will add hover shadow effect and smooth transition on Pay Now button in the polish phase.

Week 10 | Salesforce & MVC Pattern · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Studied Salesforce pages and the Model-View-Controller (MVC) pattern in Salesforce.
- Learned why Salesforce is relevant despite being older technology: it can load external JavaScript libraries freely, unlike LWC which is restricted by Lightning Locker Service.
- Understood the Salesforce bridge pattern: a VF page loads the Razorpay checkout SDK, processes payment and redirects back to the LWC portal page with result URL parameters.
- Studied , , and inline JavaScript in VF pages.

■ Project Work & Progress

Built RazorpayPayment.vfp (Visualforce page). Page reads amount, paymentId and returnUrl from URL parameters. Loads Razorpay checkout.js from CDN. Opens the payment popup automatically. On success, redirects to returnUrl with razorpay_payment_id appended as a URL parameter.

■ Challenges Faced

Razorpay popup was not opening automatically — needed to call razor.open() explicitly after defining the Razorpay object. The Razorpay constructor creates the object; open() launches the UI. Also, the test mode key must start with rzp_test_.

■ Additional Comments

The VF bridge pattern is reusable for any external JavaScript SDK. The same architecture would work for Stripe, PayU or any other payment gateway. A good pattern to document clearly in the project report.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Studied Razorpay API documentation — test mode, order creation, payment capture and handler callbacks.
- Understood the full redirect-and-return flow: LWC → redirect to VF page → VF loads Razorpay popup → payment succeeds → VF redirects back to LWC with payment ID in URL.
- Learned about Lightning Locker Service — why external JavaScript SDKs cannot be loaded inside LWC components directly.
- Practised reading URL parameters in LWC using `CurrentPageReference` from the navigation module.

■ Project Work & Progress

Built `makePayment` LWC HTML template and JavaScript controller. `connectedCallback` reads URL parameters (`razorpayPaymentId`, `paymentId`, `assessmentId`). If parameters are present, calls `confirmPayment` Apex method imperatively. Shows payment breakdown: base amount, discount or penalty and total amount.

■ Challenges Faced

Reading URL parameters in LWC requires using `CurrentPageReference`: `import { CurrentPageReference } from 'lightning/navigation'` and `@wire(CurrentPageReference)`. The `pageReference.state` object holds the parameters. Correct import was the key missing piece.

■ Additional Comments

The `connectedCallback` URL parameter reading is the critical integration point between the Razorpay VF page and the LWC. Getting this right is essential for the payment flow to work end-to-end.

Week 11 | Asynchronous Apex & Deployment · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Learned asynchronous Apex: `@future` methods, `Batchable` (large dataset processing in chunks), `Schedulable` (timed execution) and `Queueable` (chainable jobs).
- Understood governor limits and why async Apex has higher limits than synchronous Apex.
- Learned Change Sets for deployment: outbound change set, inbound change set, upload and deploy.
- Studied Salesforce DX project structure: `sfdx-project.json` and `force-app/main/default` folder layout.

■ Project Work & Progress

- Built `TaxReassessmentScheduler.cls` — a `Schedulable` Apex class that runs on April 1st every year and creates new `Tax_Assessment__c` records for all Active properties.
- Built `DueDateReminderBatch.cls` — a `Batchable` Apex class that queries tax payments with due date within 7 days and status Pending, then sends email reminders to property owners.
- Both classes include test coverage. Scheduler tested with `Test.schedule()`. Batch tested with `Database.executeBatch()` in test context.

■ Challenges Faced

Sending emails from Batch Apex requires `Messaging.sendEmail()` inside the `execute()` method. External Identity User contacts do not always have email addresses. Added a null check on email before attempting to send — prevents

NullPointerException.

■ Additional Comments

The annual reassessment scheduler is important business logic. Without it, tax assessments would only be created once at registration and never updated. Running on April 1st aligns with the Indian financial year.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Attended async Apex sessions to understand what Omkar is building for the scheduler and reminder batch.
- Studied the Salesforce deployment model — metadata API, change sets and SFDX project structure.
- Learned about LWC lifecycle hooks: `connectedCallback` (runs when component is inserted), `disconnectedCallback` (cleanup) and `renderedCallback` (after render).
- Studied when to use `@wire` vs imperative Apex calls — wire is reactive, imperative is on-demand and bypasses cache.

■ Project Work & Progress

Refined the `makePayment` LWC `connectedCallback` to correctly detect return from Razorpay. Added a loading spinner that shows while `confirmPayment` is being called. Added success screen with transaction ID display after successful payment confirmation.

■ Challenges Faced

Loading spinner needed to remain visible while the async Apex call was in flight. Used `isLoading` property set to true before the call and false in the finally block. Without finally, a failed call would leave the spinner running indefinitely.

■ Additional Comments

The finally block pattern for spinner management is a good LWC practice. Whether the Apex call succeeds or fails, the spinner should stop. Always use try-catch-finally for imperative Apex calls.

Week 12 | Integration — REST & Razorpay Gateway · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Learned Salesforce integration patterns: REST inbound, REST outbound and event-driven integration.
- Studied authentication for outbound calls: Named Credentials for storing endpoint and credentials securely.
- Learned about the two-step payment confirmation pattern: first DML saves transaction data, second DML changes status to trigger approval flow.
- Understood why a single DML combining both operations causes record-locking data loss.

■ Project Work & Progress

- Completed `MakePaymentController.cls` fully. Three methods implemented:
 - `getPaymentDetails()` — returns base amount, 2% discount if before due date, 10% penalty if overdue, and net payable.
 - `confirmPayment()` — two-step DML: first saves `Transaction_Id__c` and `Payment_Date__c`, then sets `Payment_Status__c` = Pending Verification.
 - `getPaymentStatus()` — imperative, non-cached call returning the current payment status.
- Payment flow tested end-to-end using Razorpay test card 4111111111111111. Full cycle verified successfully.

■ Challenges Faced

In early testing a single update that set both the transaction ID and the status simultaneously triggered the approval flow, locking the record before the transaction ID was saved — causing data loss. The two-step pattern resolved this completely and permanently.

■ Additional Comments

The two-step confirmation is an important finding for any Salesforce project combining external data capture with approval processes. Documented as a key architectural decision in the project report.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Tested the full Razorpay payment flow from the LWC side: Pay Now button → redirect to VF page → Razorpay popup opens → test payment → redirect back to LWC with payment parameters.
- Verified that connectedCallback correctly reads URL parameters and calls confirmPayment.
- Studied how to add the Razorpay CDN domain to Salesforce CSP Trusted Sites — required for the VF page to load the external script.
- Practised adding Trusted Site entries in Setup → Security → CSP Trusted Sites.

■ Project Work & Progress

Full payment flow working end-to-end from the portal. The property card Pay Now button correctly navigates to makePayment LWC, which redirects to VF, which opens Razorpay, which redirects back. After officer approval the dashboard shows Paid status correctly.

■ Challenges Faced

Razorpay CDN URL was missing from CSP Trusted Sites — browser blocked the script with a CSP violation error. Fixed by adding checkout.razorpay.com and api.razorpay.com to the trusted sites list. Added this check to the deployment checklist.

■ Additional Comments

Seeing the Razorpay popup appear, entering a test card, and watching the Salesforce record update automatically through the entire chain was a major milestone. The full payment cycle works correctly.

Week 13 | Modern JavaScript & LWC Foundations · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Covered ES6+ JavaScript: arrow functions, template literals, destructuring, spread operator and modules.
- Studied Promises and async/await for asynchronous operations — the pattern used for all Apex calls from LWC.
- Learned Array methods: map(), filter(), reduce(), find() — used heavily in LWC for processing Apex response data.
- Formal LWC module began: component structure, data binding with @track and @api, decorators and CSS scoping.

■ Project Work & Progress

Built PropertyOwnerDashboardController.cls fully: getDashboardSummary(), getMyProperties() with search and filter support, and getRecentPayments(). All methods use without sharing and query only records where Owner_Contact__c matches the current user's Contact.

■ Challenges Faced

Finding the current portal user's Contact ID: used UserInfo.getUserId() then queried Contact WHERE Id IN (SELECT ContactId FROM User WHERE Id = :userId). This two-query pattern is needed because User.ContactId is not directly accessible from all contexts.

■ Additional Comments

The dashboard controller is the most used Apex class. Performance matters — added LIMIT clauses on all queries to prevent governor limits. Recent payments limited to 10 records, getMyProperties limited to 50 with a separate count query for the total.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Built propertyOwnerDashboard LWC — the main dashboard component that contains all other components.
- Dashboard has three sections: Summary cards row, a search + filter toolbar and the property cards grid.
- Learned how to pass data from parent to child LWC using @api properties and receive events from child using CustomEvent.
- Studied SLDS (Salesforce Lightning Design System) grid classes for responsive layout.

■ Project Work & Progress

propertyOwnerDashboard LWC is functionally complete. Shows real data from Apex. Search bar filters by name or address. Filter tabs (All / Active / Pending / Overdue) update the displayed list. Summary cards show accurate counts. Property cards render correctly in a responsive 3-column grid.

■ Challenges Faced

Responsive grid in LWC — SLDS responsive classes: slds-size_1-of-1 slds-medium-size_1-of-2 slds-large-size_1-of-3. Understanding the breakpoint class naming convention took practice with SLDS documentation.

■ Additional Comments

The dashboard is the first impression citizens get of the portal. The responsive grid ensures it works on both desktop and mobile. Will add a skeleton loading state (animated grey placeholder boxes) while data loads.

Week 14 | LWC Advanced — Payment & eKYC Components · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Learned advanced LWC: component lifecycle, events and communication, wire service with Apex, navigation service.
- Studied LWC events in detail — bubbling vs non-bubbling events, composed events for crossing shadow DOM boundaries.
- Built EKycController.cls — manages eKYC flow: get-or-create Property_Owner__c record, save PAN and Aadhar with validation, set KYC Active flag.
- PAN validation regex: [A-Z]{5}[0-9]{4}[A-Z]{1} — enforced in Apex and as a validation rule on the object.

■ Project Work & Progress

- EKycController.cls completed and tested. Two methods: getOrCreatePropertyOwner() and saveKycDetails().
- eKycForm LWC completed — two-step form working end-to-end: personal info saved in Step 1, PAN and Aadhar validated and saved in Step 2.
- On successful eKYC completion, the component fires a CustomEvent that navigates the user to the property registration screen flow.

■ Challenges Faced

If a Property_Owner__c record already existed for the portal user (returning user), the getOrCreatePropertyOwner() method must update it rather than insert a duplicate. Added upsert with external ID to handle both new and returning

users cleanly.

■ Additional Comments

The eKYC step gates property registration — citizens must complete KYC before they can register a property. This is a real civic requirement that makes the portal authentic and not just a demo.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Completed makePayment LWC integration with the two-step payment confirmation flow.
- Learned how to navigate between portal pages using NavigationMixin from 'lightning/navigation'.
- Studied the PageReference types for Experience Cloud: comm__namedPage for standard pages, standard__webPage for external URLs.
- Practised conditional rendering of the payment success screen versus the payment initiation screen using a single isPaymentComplete flag.

■ Project Work & Progress

makePayment LWC fully working: shows payment breakdown, handles Razorpay redirect, reads return parameters, calls confirmPayment and shows success screen with transaction ID. Pay Now button correctly disabled and replaced with a success message after payment.

■ Challenges Faced

NavigationMixin.Navigate was needed to redirect to the Visualforce page since it is an external URL from the portal's perspective. Had to use standard__webPage type with the full VF page URL including the org's visualforce.com domain.

■ Additional Comments

The payment component handles four states: loading, payment pending, payment in progress (redirected to VF) and payment complete. Managing these four states cleanly with a small set of boolean properties is the key design challenge.

Week 15 | Cases, Grievances & TaxAssessment Trigger · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Built TaxAssessmentTrigger.cls — fires after insert when Tax_Assessment__c is created with Annual_Tax__c greater than zero.
- Trigger creates Tax_Payment__c with base amount, applies discount or penalty based on due date, and sets the portal user as the record owner for sharing-set access.
- Finding the portal user from the trigger context: query Property__c → Owner_Contact__c → User to find the citizen's portal user and set OwnerId.
- Built CaseController.cls — create cases linked to the portal user's contact, fetch cases with comments, add comments and return case summary counts.

■ Project Work & Progress

- TaxAssessmentTrigger.cls completed with test coverage at 86%.
- CaseController.cls completed — four @AuraEnabled methods: createCase(), getMyCases(), addComment() and getCaseSummary().
- Tested the complete chain: property approved by officer → PropertyTrigger fires → tax assessment created → TaxAssessmentTrigger fires → tax payment created with correct portal user as owner.

■ Challenges Faced

TaxAssessmentTrigger needs to set OwnerId of Tax_Payment__c to the portal user. But the running user in the trigger is the officer, not the citizen. Had to query the Contact from the property and then find the portal User with that ContactId.

■ Additional Comments

Getting the ownership chain right — assessment trigger sets citizen as payment owner, sharing set maps user contact to payment owner contact — is the foundation of the entire portal data access model. One wrong link breaks everything.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Built caseGrievance LWC — two tabs: Raise New Case and My Cases.
- Raise New Case tab: case type picklist (Billing Query, Assessment Dispute, Portal Issue, General Inquiry), priority, description and optional related property dropdown.
- My Cases tab: shows cases with status badge, date, subject and a Replies section for each case.
- Learned tabset pattern: lightning-tabset with lightning-tab elements, active-tab-value controls which tab is shown.

■ Project Work & Progress

caseGrievance LWC is functional. Citizens can raise cases. My Cases shows open and closed cases. Reply button opens a text input that submits a comment. Officer comments appear correctly in the citizen's view.

■ Challenges Faced

Loading case comments lazily when a case is expanded required an imperative Apex call triggered by a user action — not @wire, because the case ID parameter was not known until the user expanded that specific case.

■ Additional Comments

The tabset pattern for Raise vs View cases is clean UX. Citizens immediately see both the ability to act (raise a case) and their history (my cases) in one place. Status badges (Open = blue, In Progress = orange, Closed = green) give instant visual status.

Week 16 | Experience Cloud Portal Deep-Dive · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Deepened Experience Cloud knowledge: page layouts, page variants, visibility rules and audience targeting.
- Configured audience-based visibility — certain components only visible to logged-in portal users, not guest users.
- Studied Navigation Menu setup — linking portal pages and external URLs in the site navigation.
- Learned how to embed an iframe LWC in Experience Cloud for the Zone and Ward Finder external application.

■ Project Work & Progress

- All LWC components added to Experience Cloud pages using App Builder: eKycForm, propertyOwnerDashboard, makePayment, caseGrievance and iframeEmbed.
- Navigation menu configured with links to all pages. Active page detection working correctly in the navigation bar.
- Tested a simulated citizen user account end-to-end — registration, eKYC, property submission, approval, payment and grievance all working from the citizen's perspective.

■ Challenges Faced

Some LWC components were visible to guest users who had not logged in. Fixed using Audience configuration in Experience Builder — set each component's visibility to 'Users who are members of the site' only.

■ Additional Comments

Testing with a simulated citizen account is different from testing as an internal admin user. The sharing model, record visibility and available pages are all different. Always test from the citizen account, not the admin account.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Built the Zone and Ward Finder feature — a standalone HTML, CSS and JavaScript application using Leaflet.js and OpenStreetMap tiles.
- Leaflet.js renders an interactive map of Chennai with four zones as coloured polygon overlays and ward number markers.
- Users can search by area name or click the map to find their zone and ward number, which maps to Property__c.Zone__c and Property__c.Ward_Number__c fields.
- Deployed the application on Vercel (free hosting). Built an iframeEmbed LWC to embed it in the portal.

■ Project Work & Progress

Zone and Ward Finder deployed and embedded in the portal. Shows four zones (North — blue, South — green, East — orange, West — purple) with ward markers. Search returns the zone and ward number matching the property registration form fields.

■ Challenges Faced

Embedding an external URL in Salesforce via iframe requires adding the domain to CSP Trusted Sites in Setup. The Vercel deployment URL and the OpenStreetMap tile CDN URL both had to be added. Without this the browser blocks the iframe as a CSP violation.

■ Additional Comments

The Zone and Ward Finder solves a real citizen pain point — knowing which zone and ward their property falls in. It maps directly to the property registration form, reducing entry errors at registration time.

Week 17 | Agentforce & AI Assistant · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Studied foundations of AI in Salesforce: Einstein Trust Layer, data masking and zero data retention policy.
- Learned Agentforce architecture: Agents, Topics and Actions. An Agent handles a conversation, a Topic matches user intents, Actions execute the response.
- Studied Prompt Engineering: writing clear, scoped system prompts to keep the agent on topic.
- Created 15 Knowledge Articles covering Property Tax FAQs: how to calculate tax, how to pay, registration requirements, penalty policy, assessment dispute process, eKYC documents and more.

■ Project Work & Progress

- Configured the Agentforce agent in Setup: created the Agent, defined the 'Property Tax Queries' Topic with a scoped system prompt, and added a Knowledge Article Search Action.
- Ran the Einstein Trust Layer check — verified no personally identifiable information is sent to the LLM.
- Tested the agent with 15 sample citizen questions. Agent correctly answered questions about tax calculation, payment process, penalty policy and assessment disputes using the Knowledge Articles.

■ Challenges Faced

Agentforce agent was not finding Knowledge Articles correctly — the Topic was too broad. Refined the Topic description to specifically mention property tax, GCMC, assessment and payment. After refinement, AI assistant answered test questions accurately.

■ Additional Comments

Agentforce is a significant differentiator for the portal. Citizens can get instant, accurate answers to property tax questions at any time without waiting for officer office hours. The 15 articles cover the vast majority of common citizen queries.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Attended the Agentforce module sessions to understand how the AI assistant is embedded in the Experience Cloud portal.
- Added the Agentforce messaging widget to the Experience Cloud portal Home page using Experience Builder's standard Embedded Messaging component.
- Configured the widget placement in Site Settings to appear only on specific pages, not all pages.
- Completed Trailhead badge: 'Agentforce Basics'.

■ Project Work & Progress

Agentforce chat widget added to the portal Home page. Tested with 10 sample questions from a citizen perspective. Widget appears as a floating chat button. Conversations are scoped to the property tax topics correctly.

■ Challenges Faced

Agentforce widget was appearing as a floating button on all portal pages, not just the Home page. Fixed by configuring widget placement in Experience Builder's Site Settings rather than individual page settings.

■ Additional Comments

Seeing the AI assistant accurately answer a question like 'How is my property tax calculated?' using the Knowledge Articles made the portal feel like a complete product rather than just a CRUD application.

Week 18 | DevOps, Git & Salesforce DX · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Studied DevOps for Salesforce: Git with Salesforce DX, CI/CD pipeline concepts and deployment strategies.
- Learned Salesforce DX commands: sf project retrieve start, sf project deploy start, sf org list.
- Created a precise package.xml listing only the custom metadata types to retrieve a clean project folder.
- Studied branch strategy: main for stable code, develop for active development, feature branches for each component.

■ Project Work & Progress

- Pushed all project metadata to GitHub repository using Salesforce DX. README.md written documenting the org URL, setup steps and deployment instructions.
- All Apex classes, triggers, LWC components, flows, approval processes, custom objects and sharing sets are in the repository.
- Pull request workflow practised — Kaustubh reviewed Apex code, Omkar reviewed LWC code.

■ Challenges Faced

Salesforce DX metadata retrieval brings down system-generated items (standard page layouts, standard profiles) not specific to the project. Had to create a precise package.xml listing only custom metadata types for a clean repository.

■ Additional Comments

Version control is as important as the code itself. The ability to roll back to a working state saved time when a flow change broke the approval process. Always commit before making configuration changes.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Learned basic Git commands: clone, pull, add, commit, push, branch, merge and pull request workflow.
- Practised the daily development workflow: git pull → make changes → git add → commit with meaningful message → push → pull request.
- Reviewed Omkar's backend code in a pull request — checked logic and left one comment about a missing null check in the portal user query.
- Created a custom static resource CSS file for shared styles used across multiple LWC components.

■ Project Work & Progress

All LWC component code organised on GitHub with proper commits. Created a sharedStyles.css static resource with common styles — badge colours, button styles, card shadows. This replaced scattered inline styles added component by component over the weeks.

■ Challenges Faced

Merge conflict when both Omkar and I modified the DashboardController metadata at the same time. Resolved manually using VS Code's merge conflict tool which shows both versions side by side. Kept both changes.

■ Additional Comments

The pull request review caught a Thymeleaf-style error in a Thymeleaf layout tag — actually an LWC `lwc:if` syntax issue — that would have caused a render failure. Two people checking code is always better than one.

Week 19 | Testing, Bug Fixes & Code Cleanup · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Software / Skills / Tools Learnt

- Wrote comprehensive Apex test classes for all five controllers and both triggers.
- Learned test data creation patterns: using `@TestSetup` to create common test records once for all test methods in the class.
- Studied mock testing for Apex controllers — using `Test.setMock()` was not needed here, but understood the concept for future REST callout testing.
- Ran all test classes in the org — all passing with 82% average code coverage across the project.

■ Project Work & Progress

- Found and fixed three bugs during testing:
 - Bug 1: Tax Assessment Trigger was not setting OwnerId correctly when the property had no associated portal user — added null check.
 - Bug 2: `confirmPayment` was failing silently when called twice (double-click on Pay Now) — added an idempotency check: if payment status is already Pending Verification, skip the second DML.
 - Bug 3: `getDashboardSummary` was counting overdue payments that belonged to other citizens — the SOQL WHERE clause was missing the Owner.Contact filter.

■ Challenges Faced

Bug 3 (dashboard showing other citizens' overdue counts) was a data security issue, not just a UI bug. The SOQL query was counting all Pending payments past due date globally. Adding the correct WHERE filter fixed both the security issue and the wrong count simultaneously.

■ Additional Comments

Testing with multiple citizen accounts simultaneously exposed the data isolation bugs. Always test the portal with at least two separate citizen accounts to verify data isolation. Single-account testing misses cross-account data leakage bugs.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Software / Skills / Tools Learnt

- Conducted full manual end-to-end test of every feature in the portal from the citizen's perspective.
- Used browser Developer Tools to test at different screen sizes — mobile (375px), tablet (768px) and desktop (1280px).
- Found three UI bugs: property cards grid was showing three columns on mobile, the eKYC Step 2 button label was wrong and the overdue badge colour was not consistent.
- Fixed all three: corrected SLDS responsive classes, fixed button label and standardised badge colour in the shared CSS.

■ Project Work & Progress

All UI bugs fixed. App tested on mobile view — property cards now stack to a single column on small screens. Badge colours consistent across all components. All portal pages tested and working. Created a bug report document listing all issues found and their fixes.

■ Challenges Faced

Mobile property card grid was showing three columns instead of one on small screens because of a typo in the SLDS class: slds-small-size_1-of-1 does not exist. Correct class is slds-size_1-of-1 which applies at all breakpoints. One character difference, significant visual impact.

■ Additional Comments

Manual testing from the citizen's perspective reveals things automated tests miss — visual layout issues, confusing labels and navigation dead-ends. Both automated and manual testing are necessary for a quality product.

Week 20 | Consolidation & MFRP Final · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Skills Applied

- Full consolidation week — integrated all modules, resolved remaining issues and prepared the MFRP Final demo.
- Ran a complete regression test: eKYC → property registration → approval → tax assessment → payment → dashboard → grievance → AI assistant — all passing.
- Added Knowledge Articles for the Agentforce AI assistant (refined to 15 total). Verified each article is returned correctly by the agent for the corresponding citizen question.
- Refined all Apex test classes to achieve 82% average coverage. Ran the complete test suite — all passing.

■ Project Work & Progress

- All Apex classes completed: PropertyTrigger, TaxAssessmentTrigger, MakePaymentController, PropertyOwnerDashboardController, EKycController, CaseController, TaxReassessmentScheduler, DueDateReminderBatch.

- All approval processes, flows and record-triggered automations tested end-to-end.
- MFRP Final demo prepared: 20-minute walkthrough covering the complete citizen journey from eKYC to AI assistant.
- MFRP Final review: SME panel rated the project highly, noting the two-step payment confirmation and Agentforce integration as standout decisions.

■ Challenges Faced

Agentforce agent was not finding Knowledge Articles for very short queries like 'penalty'. Refined the Topic prompt to mention that the agent should search Knowledge even for short or ambiguous queries. Also created more specific article titles to improve search relevance.

■ Additional Comments

The MFRP Final review confirmed the project is on track. Mentor specifically noted the two-step payment confirmation as a mature architectural decision and the Agentforce integration as a strong differentiator for a final-year project.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Skills Applied

- Completed all LWC components: eKycForm, propertyOwnerDashboard, propertyCard, makePayment, caseGrievance, iframeEmbed and the Agentforce widget configuration.
- Conducted final UI polish: consistent spacing, SLDS token-based colours, hover effects on property cards and smooth transitions on status badge changes.
- All portal pages added to Experience Cloud navigation. Tested navigation from every page to every other page.

■ Project Work & Progress

Full portal UI tested and polished. All six citizen-facing pages working: Home (Dashboard), eKYC, Register Property, Make Payment, Zone & Ward Finder and Raise Grievance. The Agentforce chat widget is accessible from the Home page.

■ Challenges Faced

During the MFRP Final demo, a CSP error appeared for the Zone & Ward Finder iframe — a Vercel re-deployment had changed the URL prefix slightly. Fixed by updating the iframeEmbed LWC URL and the CSP Trusted Sites entry. Added the production URL to the deployment checklist.

■ Additional Comments

MFRP Final went well. Panel specifically noted the colour-coded status badges and the responsive grid as strong UX decisions. Fourteen weeks of progressive component building produced a cohesive, professional portal.

Week 21 | Final Evaluation & Project Demonstration · PROJECT PHASE

● Omkar — Salesforce Backend Developer | Apex / LWC / Admin

■ Notes — Final Summary

- Conducted complete end-to-end regression test of all features before the final evaluation.
- All Apex test classes passing with 82% average code coverage across all classes.
- Modules delivered: eKYC verification, property registration with two-level approval, BSR-based tax assessment creation, Razorpay payment with two-step confirmation, real-time citizen dashboard, cases and grievances, zone and ward finder, Agentforce AI assistant, scheduled annual reassessment and due-date email reminders.
- Final project report completed and submitted to both college and industry mentors.

■ Project Work & Progress

- Final project demonstration presented to the mentor and SME panel.

- Demo sequence: (1) Register new portal user → (2) Complete eKYC → (3) Register property via screen flow → (4) Officer approves → (5) PropertyTrigger creates tax assessment → (6) TaxAssessmentTrigger creates payment record → (7) Citizen pays via Razorpay → (8) Officer verifies payment → (9) Dashboard shows Paid → (10) Citizen raises grievance → (11) AI assistant answers property tax query.
- Final evaluation result: Completed successfully. Report submitted.

■ Challenges Faced

During the final demo the Razorpay popup did not appear initially — the org's CSP settings had been reset during a metadata deployment earlier that morning. Fixed by re-adding the Razorpay CDN URL to trusted sites. Added a pre-demo CSP verification to the demo checklist.

■ Additional Comments

- 21 weeks from no Salesforce knowledge to a working municipal tax management portal with payment integration and an AI assistant.
- Key learnings: declarative-first design saves time and reduces Apex complexity; the two-step DML pattern is essential for approval-process safety; sharing sets are the entire foundation of portal data access; and Visualforce remains valuable as a bridge for external JavaScript SDKs.
- Future enhancements identified: pro-rated tax for mid-year registrations, monthly compounding penalty at 1.5% per month, multi-channel SMS and WhatsApp notifications, and a mobile progressive web application.

● Kaustubh — Frontend + Integration | LWC / UI / Experience Cloud

■ Notes — Final Summary

- All LWC components deployed and working on the Experience Cloud portal.
- Components delivered: eKycForm, propertyOwnerDashboard, propertyCard, makePayment, caseGrievance, iframeEmbed and Agentforce chat widget.
- Portal navigation fully configured with correct page links and audience-based component visibility.
- CSS is consistent across all components using SLDS design tokens and a shared static resource for custom overrides.

■ Project Work & Progress

- Participated in the final evaluation demo. Demonstrated all frontend features live in sequence.
- Presented the dashboard as the first screen — immediately shows the citizen the full picture of their properties and tax status.
- Panel noted the colour-coded status badges, the responsive grid layout and the Agentforce chat widget as strong UX and product decisions.

■ Challenges Faced

During final testing discovered the mobile view of property cards was showing three columns instead of one on very small screens due to a leftover SLDS class typo. Fixed the SLDS responsive class in propertyCard LWC. Small fix with large visual impact on mobile devices.

■ Additional Comments

- Frontend work was deeply intertwined with Omkar's backend — every LWC component depends on a specific Apex controller method and its return structure. Regular communication about wrapper class field names, return types and error formats was as important as the coding itself.
- 21 weeks gave a complete picture of the Salesforce full-stack development process — from data model design to production-grade UI delivered to a citizen portal.
- The internship confirmed that a declarative-first, code-where-necessary approach on Salesforce can deliver a maintainable and extensible product within the time budget of a final-year industry internship.